VÕ TIẾN
Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

Cấu Trúc Dữ Liệu và Giải Thuật (DSA)

DSA - HK243

Cuối Kì

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

TP. HỒ CHÍ MINH, THÁNG 7/2025

Mục lục

1 Lý thuyết cơ bản Heap **2**

2 Câu hỏi trong các đề thi **3**

1 Lý thuyết cơ bản Heap

Định nghĩa và tính chất

- **Heap** là một cây nhị phân *gần hoàn chỉnh*, tức là:
  - Mọi tầng đều được điền đầy từ trái qua phải, trừ tầng cuối cùng.
  - Tầng cuối cùng có thể chưa đầy nhưng các node vẫn được xếp từ trái sang phải.
- **Phân loại Heap:**
  - **Min-Heap**: Mỗi node cha có giá trị nhỏ hơn hoặc bằng node con.
  - **Max-Heap**: Mỗi node cha có giá trị lớn hơn hoặc bằng node con.

Các công thức quan trọng

- **Chiều cao của Heap** (với n phần tử):

$$h = \lfloor \log_2 n \rfloor + 1$$

- **Số phần tử trong Heap có chiều cao** h:

$$2^{h-1} \leq n \leq 2^h - 1$$

- Nếu Heap là n**-ary heap** (mỗi node có tối đa n con), các công thức trên sẽ thay đổi tương ứng.

Độ phức tạp các thao tác trên Heap

- **Heapify:** $O(\log n)$
- **Build-Heap:** $O(n)$
- **Insert và Remove:** $O(\log n)$
- **Heap Sort:** $O(n \log n)$ (trong cả 3 trường hợp: tốt, xấu, trung bình)

Cách làm bài toán Insert và Remove hiệu quả

- **Nên vẽ cây thay vì vẽ mảng**, vì hình ảnh trực quan sẽ giúp thao tác nhanh hơn.
- **Cách vẽ cây từ một mảng Heap:**
  1. Sắp xếp các phần tử theo từng tầng, từ trái qua phải.
  2. Tầng đầu tiên chứa root, tầng tiếp theo chứa 2 node con của root, tầng kế tiếp chứa 4 node con của 2 node trên, v.v.
- **Cách thực hiện thao tác Insert và Remove:**
  1. **Insert (Thêm phần tử vào Heap)**
     - Thêm vào vị trí cuối cùng của cây (node lá ngoài cùng bên phải).
     - Thực hiện **Reheap Up** (đẩy giá trị lên trên nếu vi phạm tính chất Heap).
  2. **Remove (Xóa phần tử gốc - root)**
     - Swap phần tử root với phần tử cuối cùng.
     - Xóa phần tử cuối cùng (vốn là phần tử root ban đầu).
     - Thực hiện **Reheap Down** (đẩy giá trị xuống dưới nếu vi phạm tính chất Heap).

Lưu ý khi làm bài tập code

- Cần nắm vững các thuật toán **Reheap Up, Reheap Down, Build Heap, và Heap Sort**.
- Thực hành vẽ cây trước khi viết code để dễ hình dung các bước thao tác.

2 Câu hỏi trong các đề thi

1. Heap là gì trong cấu trúc dữ liệu?

   a) Một cây nhị phân tìm kiếm có các khóa được sắp xếp theo thứ tự tăng dần từ trái sang phải.
   b) Một cây nhị phân đầy đủ (complete binary tree) thoả mãn tính chất phân bố: mỗi nút cha nhỏ hơn hoặc bằng (hoặc lớn hơn hoặc bằng) các nút con của nó.
   c) Một danh sách liên kết nơi các phần tử được sắp xếp theo thứ tự tăng dần.
   d) Một cây nhị phân mà mỗi nút có tối đa hai con và không có ràng buộc về thứ tự.

   Đáp Án: B

2. Cho một mảng gồm 9 phần tử $[a_1, a_2, \ldots, a_9]$. Giả sử sau quá trình xây dựng thành một ta thu được mảng:

$$[a_9, a_6, a_3, a_4, a_7, a_1, a_2, a_5, a_8]$$

   Dãy nào dưới đây có thể là kết quả sắp xếp tăng dần của mảng ban đầu?

   a) $[a_9, a_6, a_7, a_3, a_4, a_2, a_1, a_8, a_5]$
   b) $[a_9, a_3, a_1, a_2, a_4, a_5, a_6, a_7, a_8]$
   c) $[a_9, a_6, a_7, a_3, a_1, a_2, a_5, a_4, a_8]$
   d) $[a_9, a_3, a_6, a_1, a_4, a_2, a_7, a_5, a_8]$

   Đáp Án: A - Đảm bảo node con không nằm trước node cha là được

```
            a_9
          /     \
      a_6         a_3
     /   \       /   \
   a_4   a_7   a_1   a_2
  /   \
a_5   a_8
```

3. Cho một mảng gồm 10 phần tử $[b_1, b_2, \ldots, b_{10}]$. Giả sử sau quá trình xây dựng thành một **3-ary max-heap** (mỗi nút có tối đa 3 con), ta thu được mảng:

$$[b_2, b_4, b_1, b_6, b_7, b_5, b_8, b_9, b_{10}, b_3]$$

   Dãy nào dưới đây có thể là kết quả sắp xếp giảm dần của mảng ban đầu?

   a) $[b_2, b_4, b_7, b_5, b_1, b_9, b_8, b_{10}, b_6, b_3]$
   b) $[b_2, b_1, b_3, b_5, b_4, b_6, b_7, b_8, b_9, b_{10}]$
   c) $[b_2, b_3, b_6, b_1, b_4, b_5, b_7, b_8, b_9, b_{10}]$
   d) $[b_4, b_5, b_3, b_1, b_2, b_6, b_7, b_8, b_9, b_{10}]$

   Đáp Án: A – Đảm bảo node con không nằm trước node cha là được

   Cấu trúc cây 3-ary từ mảng trên:

```
                b_2
          /      |      \
        b_4      b_1    b_6
    /    |    \  /  |  \
  b_7   b_5  b_8 b_9 b_10 b_3
```

4. Biết rằng mảng A dùng để biểu diễn một maxheap và một chỉ số i. Biết rằng việc truy cập các phần tử trong mảng đều hợp lệ, biểu thức nào sau đây LUÔN ĐÚNG?

   a) A[i] > A[2i + 1]
   b) A[i] < A[2i + 2]
   c) A[i] > A[2i]
   d) A[i] < A[2i − 1]

   Đáp Án: A - Trong maxheap, phần tử tại chỉ số i luôn lớn hơn hoặc bằng phần tử con bên phải ($A[2i+1]$).

5. Cho một min-heap được biểu diễn dưới dạng array như sau: 1, 4, 7, 8, 10, 12, 13 Đâu là một biểu diễn ĐÚNG cho trạng thái của minheap sau khi xóa một phần tử ra khỏi min-heap

   a) 4, 7, 8, 10, 13, 12
   b) 4, 7, 8, 13, 10, 12
   c) 4, 8, 13, 7, 10, 12
   d) 4, 8, 7, 13, 10, 12

Đáp Án: A -

6. Đâu là độ phức tạp của thao tác ReHeapUp?

   a) O(logn) b) O(n) c) O(nlogn) d) O(1)

   Đáp Án: A - ReHeapUp có độ phức tạp $O(\log n)$.

7. Đâu là độ phức tạp của thao tác build heap?

   a) O(logn) b) O(n) c) O(nlogn) d) O(1)

   Đáp Án: B - Build heap có thể thực hiện trong $O(n)$ với thuật toán bottom-up.

8. Đâu là độ phức tạp của heap sort?

   a) O(logn) b) O(n) c) O(nlogn) d) O(1)

   Đáp Án: C - Heap sort có độ phức tạp $O(n \log n)$.

9. Đâu là độ phức tạp của thao tác xóa 1 phần tử?

   a) O(logn) b) O(n) c) O(nlogn) d) O(1)

   Đáp Án: A - Xóa 1 phần tử trong heap có độ phức tạp $O(\log n)$.

10. Đâu là độ phức tạp của giải thuật Heapsort ở trường hợp tốt nhất (best-case)

    a) O(logn) b) O(n) c) O(nlogn) d) $O(n^2)$

    Đáp Án: C - Best-case của heap sort cũng là $O(n \log n)$.

11. Cho max-heap mang các giá trị số nguyên như sau [29, 20, 10, 15, 18, 9, 5, 13, 2, 4, 15]. Lần lượt lấy m phần tử từ đỉnh của max-heap nói trên để tổng của m phần tử này không vượt quá 70. Hãy cho biết giá trị của m.

    a) 2 b) 3 c) 4 d) 5

    Đáp Án: B - Tổng 3 phần tử đầu tiên là $29 + 20 + 18 = 67 < 70$.

12. Với hai max heap có kích thước n, độ phức tạp thời gian tối thiểu có thể có để tạo một maxheap từ các phần tử của hai maxheap là bao nhiêu?

    a) O(logn) b) O(n) c) O(nlogn) d) $O(n^2)$

    Đáp Án: B - Gộp 2 heap rồi build heap mới tốn $O(n)$.

13. Trong một maxheap nhị phân chứa n phần tử, phần tử nhỏ nhất có thể được tìm thấy trong thời gian

    a) O(logn) b) O(n) c) O(nlogn) d) $O(n^2)$

    Đáp Án: B - Phần tử nhỏ nhất có thể nằm ở leaf $\rightarrow$ duyệt $O(n)$.

14. Một min-heap nhị phân hoàn chỉnh được tạo bằng cách bao gồm mỗi số nguyên trong [1, 1023] đúng một lần. Độ sâu của một nút trong heap là độ dài của đường dẫn từ gốc của heap đến nút đó. Do đó, gốc ở độ sâu 0. Độ sâu tối đa mà số nguyên 9 có thể xuất hiện là

    a) 6 b) 7 c) 8 d) 9

    Đáp Án: C - **Giải thích:** Một min-heap nhị phân hoàn chỉnh với các phần tử từ 1 đến 1023 có tổng cộng 1023 nút. Đây là một cây nhị phân hoàn chỉnh có:

    $$n = 2^h - 1 \Rightarrow 1023 = 2^{10} - 1 \Rightarrow \text{chiều cao của heap là } 9$$

    Vì gốc có độ sâu 0, nên độ sâu lớn nhất (tức là ở tầng đáy) là:

    $$9 - 1 = 8$$

Để tìm độ sâu tối đa mà số 9 có thể xuất hiện, ta cần đặt nó càng thấp càng tốt trong cây, tức là tại tầng đáy. Vì heap là **min-heap**, nên các giá trị nhỏ hơn luôn phải ở trên cha của nó. Do đó, để số 9 có thể nằm ở tầng đáy (độ sâu 8), tất cả các số từ 1 đến 8 phải nằm ở tổ tiên hoặc nút phía trên nó trong cây.

15. Cho một mảng gồm 7 biến $[x_0, \ldots, x_6]$. Giả sử sau quá trình chuyển mảng trên thành min-heap ta được mảng: $[x_6, x_4, x_2, x_3, x_1, x_5, x_0]$. Mảng nào sau đây có thể là kết quả sắp xếp tăng dần của mảng trên?

    a) $[x_6, x_1, x_2, x_3, x_4, x_5, x_0]$
    b) $[x_6, x_2, x_5, x_0, x_3, x_4, x_1]$
    c) $[x_6, x_4, x_3, x_2, x_5, x_0, x_1]$
    d) $[x_6, x_4, x_1, x_3, x_5, x_2, x_0]$

    Đáp án: C – Giải thích:

    Quá trình tạo **min-heap** từ mảng ban đầu không thay đổi các giá trị, chỉ thay đổi vị trí. Ta được min-heap:

    $$[x_6, x_4, x_2, x_3, x_1, x_5, x_0]$$

    Điều này có nghĩa là:

    $$x_6 < x_4, x_6 < x_2, \quad x_4 < x_3, x_4 < x_1, \quad x_2 < x_5, x_2 < x_0$$

16. Cho max-heap: 90, 73, 41, 25, 36, 17, 1, 2, 3, 19, 26, 7. Trạng thái heap sau khi xoá 36 khỏi heap:

    a) 90, 73, 41, 25, 26, 17, 1, 2, 3, 7, 19
    b) 90, 73, 41, 25, 19, 17, 1, 2, 3, 7, 26
    c) 90, 73, 41, 25, 26, 17, 1, 2, 3, 19, 7
    d) 90, 73, 41, 25, 19, 17, 1, 2, 3, 26, 7

    Đáp án: C – Giải thích: Đầu tiên, ta vẽ max-heap ban đầu:

```
            90
          /     \
        73       41
      /   \     /   \
    25     36  17     1
   / \    /  \  |
  2   3  19  26 7
```

    Thay giá trị 36 bằng phần tử cuối cùng là 7.

```
            90
          /     \
        73       41
      /   \     /   \
    25      7  17     1
   / \    /  \
  2   3  19  26
```

    Gọi `heapify` xuống từ vị trí 4 (nơi có 7).

```
            90
          /     \
        73       41
      /   \     /   \
    25     26  17     1
   / \    /  \
  2   3  19  7
```

17. Cho một mảng: 1, 3, 5, 4, 6, 11, 10, 8, 7, 13, 15. Đâu là mảng đầu ra đại diện cho kết quả sau hai bước heapify mảng trên thành max heap với giải thuật $O(N)$?

    a) $[15, 13, 11, 8, 6, 5, 10, 4, 7, 3, 1]$
    b) $[1, 15, 11, 8, 13, 5, 10, 4, 7, 3, 6]$
    c) $[1, 3, 5, 8, 15, 11, 10, 4, 7, 13, 6]$
    d) Các đáp án khác đều sai

Đáp án: **C** – **Giải thích:** Khi xây dựng Max Heap bằng giải thuật `bottom-up heapify` có độ phức tạp $O(n)$:

Duyệt các node từ $\lfloor n/2 \rfloor - 1$ về 0 để gọi `heapify()`.

Với mảng kích thước $n = 11$, bắt đầu từ vị trí 4 trở về 0.

Bước 1: Heapify từ index 4 → subtree là: `6, 13, 15` → sau khi heapify → `15` lên top.

$$\text{Mảng sau bước 1: } [1, 3, 5, 4, 15, 11, 10, 8, 7, 13, 6]$$

Bước 2: Tiếp tục heapify tại index 3: subtree là `4, 8, 7` → max là 8, đưa lên trên:

$$\text{Mảng sau bước 2: } [1, 3, 5, 8, 15, 11, 10, 4, 7, 13, 6]$$

18. Cho một mảng: $[10, 7, 11, 5, 4, 13, 1, 2]$. Đâu là mảng đầu ra đại diện cho min heap được tạo ra bằng cách thêm vào từng phần tử của mảng trên?

    a) $[1, 2, 4, 5, 7, 13, 11, 10]$
    b) $[1, 2, 5, 4, 7, 13, 11, 10]$
    c) $[1, 2, 5, 4, 7, 11, 13, 10]$
    d) $[1, 2, 4, 5, 7, 10, 11, 13]$

    Đáp án: **A** – **Giải thích:** Chúng ta tạo **min-heap** bằng cách thêm từng phần tử theo thứ tự vào heap và thực hiện `reheap-up` mỗi lần thêm:

    - Thêm 10 → `[10]`
    - Thêm 7 → `[7, 10]` (vì $7 < 10$, swap)
    - Thêm 11 → `[7, 10, 11]`
    - Thêm 5 → `[7, 10, 11, 5]` → `[5, 7, 11, 10]` (5 reheap-up với 10 và 7)
    - Thêm 4 → `[5, 7, 11, 10, 4]` → `[4, 5, 11, 10, 7]` (4 reheap-up với 7 và 5)
    - Thêm 13 → `[4, 5, 11, 10, 7, 13]`
    - Thêm 1 → `[4, 5, 11, 10, 7, 13, 1]` → `[1, 5, 4, 10, 7, 13, 11]` (1 reheap-up với 11 và 4)
    - Thêm 2 → `[1, 5, 4, 10, 7, 13, 11, 2]` → `[1, 2, 4, 5, 7, 13, 11, 10]` (2 reheap-up với 10 và 5)

19. Cho min-heap: $[1, 2, 6, 4, 7, 13, 11, 10]$. Đâu là min-heap đầu ra nếu ta thêm vào mảng trên phần tử có giá trị 3?

    a) $[1, 2, 3, 4, 7, 13, 11, 10, 6]$
    b) $[1, 2, 6, 3, 7, 13, 11, 10, 4]$
    c) $[1, 2, 6, 4, 7, 13, 11, 10, 3]$
    d) $[1, 2, 6, 4, 3, 13, 11, 10, 7]$

    Đáp án: B - Ban đầu, cây min-heap tương ứng là:

```
            1
         /     \
       2         6
     /   \     /   \
    4     7   13   11
   /
 10
```

Khi thêm phần tử 3, **reheap-up** thay thế node 4

```
            1
         /     \
       2         6
     /   \     /   \
    3     7   13   11
   /  \
 10    4
```

20. Phát biểu nào sau đây là đúng với một heap có độ cao h:

    a) Phần tử có key lớn nhất sẽ là root của heap
    b) Các phần tử leaf của heap chỉ nằm ở độ cao h
    c) Heap là một cây nhị phân hoàn chỉnh (complete binary tree)
    d) Các phát biểu khác đều đúng

    Đáp án: D

    - **A sai:** Phần tử có key lớn nhất chỉ nằm ở root nếu là **max-heap**. Trong **min-heap** thì root là phần tử nhỏ nhất. Do đó, phát biểu này không đúng với mọi heap.
    - **B sai:** Trong heap (là cây nhị phân hoàn chỉnh), các leaf có thể nằm ở độ cao h hoặc $h-1$, không nhất thiết tất cả đều ở cùng một độ cao h.
    - **C đúng:** Heap luôn có cấu trúc là **cây nhị phân hoàn chỉnh** (complete binary tree), tức là tất cả các tầng đều đầy đủ trừ có thể tầng cuối cùng và các node ở tầng cuối cùng được lấp từ trái sang phải.
    - **D sai:** Vì chỉ có phát biểu C đúng nên "Các phát biểu khác đều đúng" là sai.

21. Độ phức tạp của các giải thuật xây dựng heap như heapify và bottom-up-build tương ứng là:

    a) $O(\log_2 n)$ và $O(n)$
    b) $O(n)$ và $O(\log_2 n)$
    c) $O(n)$ và $O(n)$
    d) $O(\log_2 n)$ và $O(\log_2 n)$

    Đáp án: A - **Giải thích:**

    - `heapify(i)` có độ phức tạp là $O(\log_2 n)$ vì nó đi từ node i xuống lá trong cây nhị phân.
    - `buildHeap()` theo cách bottom-up có độ phức tạp là $O(n)$, do việc gom nhiều lần `heapify` cho các node ở nửa đầu mảng nhưng phần lớn các node nằm gần đáy nên mỗi `heapify` có chi phí nhỏ.

22. Với chức năng trả về vị trí của đỉnh có trọng số nhỏ nhất và sau đó loại bỏ đỉnh này khỏi danh sách ở mỗi lần lặp, nên áp dụng cấu trúc dữ liệu nào cho hàm minDistance thì thích hợp nhất để giảm độ phức tạp thời gian thực thi?

    a) Đống (Heap)
    b) Chồng (Stack)
    c) Danh sách liên kết đơn (Singly Linked List)
    d) AVL

    Đáp án: A - **Giải thích:** Để tối ưu hàm `minDistance` – vốn cần truy xuất đỉnh có trọng số nhỏ nhất và loại bỏ nó – thì nên dùng cấu trúc **Min-Heap**. Cấu trúc này giúp truy xuất và cập nhật giá trị nhỏ nhất với độ phức tạp $\mathcal{O}(\log V)$, thay vì $\mathcal{O}(V)$ nếu dùng mảng hay danh sách liên kết. Do đó, Heap là lựa chọn tối ưu nhất.

23. Cho một 3-ary max heap: $27, 21, 15, 18, 19, 11, 9, 10, 12, 14, 8$. Trạng thái heap sau khi xoá phần tử 19 là:

    a) $27, 21, 15, 18, 14, 11, 9, 10, 12, 8$
    b) $27, 21, 15, 18, 12, 11, 9, 10, 14, 8$
    c) $27, 21, 15, 18, 8, 11, 9, 10, 14, 12$
    d) $27, 21, 15, 18, 8, 11, 9, 10, 12, 14$

    Đáp án: D - **Giải thích:** Ta có một 3-ary Max Heap:

    ```
    27, 21, 15, 18, 19, 11, 9, 10, 12, 14, 8
    ```

    Dạng cây:

    ```
                27
          /      |      \
        21       15      18
      / | \     / | \     |
    19 11  9 10 12 14    8
    ```

Xóa phần tử 19: swap với giá trị 8 (và 8 sẽ reheap lên mà $21 > 8$ nên bỏ qua)

```
            27
        /    |    \
      21     15     18
    / | \   / | \
   8 11  9 10 12 14
```

24. Cho một 3-ary max heap ban đầu: $30, 20, 15, 18, 19, 11, 10$. Trạng thái heap sau khi chèn thêm các phần tử 25 và 5 là:

    a) $30, 25, 15, 20, 19, 11, 10, 18, 5$
    b) $30, 25, 15, 18, 19, 11, 10, 20, 5$
    c) $30, 20, 25, 18, 19, 11, 10, 15, 5$
    d) $30, 20, 15, 18, 19, 11, 10, 25, 5$

    Đáp án: C - **Giải thích:** Ta có một 3-ary Max Heap ban đầu:

```
30, 20, 15, 18, 19, 11, 10
```

    Dạng cây:

```
        30
      /  |  \
    20   15   18
   / | \
 19 11 10
```

    Chèn 25: Thêm vào cuối cây, dưới node 15. Vì $25 > 15$ nên đổi chỗ với 15.

```
        30
      /  |  \
    20   25   18
   / | \  |
 19 11 10 15
```

    Chèn 5: Thêm vào cuối cây, dưới node 25. Vì $5 < 25$ nên không đổi chỗ.

```
        30
      /  |  \
    20   25   18
   / | \  | \
 19 11 10 15 5
```

    Dạng mảng sau cùng:

```
30, 20, 25, 18, 19, 11, 10, 15, 5
```

    Dữ liệu sau cho 2 câu tiếp theo:

```cpp
class Heap {
    int *harr; // pointer to array of elements in heap
    int heap_size; // Current number of elements.
public:
    bool isMinHeap() {
        // This function will return true if given level order traversal is Min Heap
        for (int i = (heap_size / 2 - 1); i >= 0; i--) {
            if (/*Code1*/)
                return false;
            if (/*Code2*/) {
                if (harr[i] > harr[2 * i + 2])
                    return false;
            }
        }
```

```
        return true;
    }
};
```

25. Hãy điền vào `Code1`:

    a) harr[i] < harr[2 * i + 1]
    b) harr[i] > harr[2 * i + 2]
    c) harr[i] < harr[2 * i + 2]
    d) harr[i] > harr[2 * i + 1]

    Đáp án: D - **Giải thích:** Với Min-Heap, mỗi nút cha phải nhỏ hơn hoặc bằng các con của nó. Con trái có chỉ số là `2 * i + 1`, nên nếu `harr[i] > harr[2 * i + 1]` thì vi phạm tính chất Min-Heap. Điều kiện cần kiểm tra là `harr[i] > harr[2 * i + 1]`.

26. Hãy điền vào `Code2`:

    a) 2*i + 1 < heap_size
    b) 2*i + 2 < heap_size
    c) 2*i + 2 <= heap_size
    d) 2*i + 1 == heap_size

    Đáp án: B - **Giải thích:** Không phải mọi nút đều có con phải. Do đó, trước khi so sánh với con phải tại `2 * i + 2`, cần kiểm tra điều kiện tồn tại: `2 * i + 2 < heap_size`. Nếu điều kiện đúng thì mới thực hiện so sánh tiếp theo.

    Dữ liệu sau cho 2 câu tiếp theo:

```
class Heap {
    int *harr; // pointer to array of elements in heap
    int heap_size; // Current number of elements.
public:
    void buildHeap() {
        // This function will build a heap from the array
        for (int i = (heap_size / 2 - 1); i >= 0; i--) {
            heapify(i);
        }
    }

    void heapify(int i) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int smallest = i;

        if (left < heap_size && /*Code1*/)
            smallest = left;
        if (right < heap_size && harr[right] < harr[smallest])
            smallest = right;
        if (smallest != i) {
            std::swap(harr[i], harr[smallest]);
            /*Code2*/
        }
    }
};
```

27. Hãy điền vào `Code1`:

    a) harrharr[left] < harr[smallest]
    b) harr[left] < harr[smallest + 1]
    c) harr[left] > harr[smallest]
    d) harr[left] > harr[smallest + 1]

    Đáp án: A - **Giải thích:** Để duy trì Min-Heap, ta cần kiểm tra xem `harr[left]` có nhỏ hơn `harr[smallest]` hay không. Nếu đúng, ta cập nhật `smallest = left` để tiếp tục so sánh với `right`. Vì vậy điều kiện đúng là: `harr[left] < harr[smallest]`.

28. Hãy điền vào `Code2`:

a) heapify(left);
b) heapify(right);
c) heapify(smallest);
d) heapify(smallest - 1);

Đáp án: C - **Giải thích:** Sau khi hoán đổi phần tử tại vị trí i với phần tử nhỏ nhất (ở vị trí left hoặc right), ta cần đệ quy gọi lại `heapify(smallest)` để đảm bảo cây con tại `smallest` vẫn thỏa mãn tính chất Min-Heap.

Dữ liệu sau cho 2 câu tiếp theo:

```cpp
class Heap {
    int *harr; // pointer to array of elements in heap
    int heap_size; // Current number of elements.
public:
    void buildHeap() {
        // This function will build a heap from the array using reheap up
        for (int i = 1; i < heap_size; i++) {
            reheapUp(i);
        }
    }

    void reheapUp(int i) {
        int parent = /*Code2*/;

        for (; i > 0 && /*Code1*/; i = parent) {
            std::swap(harr[i], harr[parent]);
            parent = (i - 1) / 2;
        }
    }
};
```

29. Hãy điền vào `Code1`:

a) harr[parent] > harr[i + 1]
b) harr[parent] > harr[i - 1]
c) harr[parent] < harr[i]
d) harr[parent] < harr[i + 1]

Đáp án: C - **Giải thích:** Hàm `reheapUp` dùng để xây dựng Max-Heap bằng cách đưa phần tử ở vị trí i lên đúng vị trí nếu nó lớn hơn cha. Do đó, điều kiện trong vòng lặp là: `harr[parent] < harr[i]`.

30. Hãy điền vào `Code2`:

a) (i - 1) / 2
b) (i + 1) / 2
c) i / 2
d) i - 1

Đáp án: A - **Giải thích:** Với cây nhị phân dạng mảng, chỉ số của cha của phần tử tại chỉ số i là: `(i - 1) / 2`. Điều này đúng với quy ước chỉ số bắt đầu từ 0.

Dữ liệu sau cho 2 câu tiếp theo:

```cpp
class Heap {
    int *harr; // pointer to array of elements in heap
    int heap_size; // Current number of elements.

public:
    // Constructor to initialize heap
    Heap(int arr[], int n) {
        // TODO
    }

    void heapSort() {
        // Build a max heap
        for (int i = /*Code1*/; i >= 0; i--) {
            heapify(i);
```

```
        }

        // Extract elements from heap one by one
        for (int i = heap_size - 1; i > 0; i--) {
            std::swap(harr[0], harr[i]);
            heap_size--;
             /*Code2*/
        }
    }

    // To maintain max-heap property
    void heapify(int i) {
         // TODO
    }
};
```

31. Hãy điền vào `Code1`:

    a) heap_size / 2 - 1 b) heap_size + 1 c) heap_size / 2 - 2 d) heap_size * 2

    Đáp án: A - **Giải thích:** Khi xây dựng Max-Heap, chỉ cần gọi `heapify` cho các nút không phải lá. Trong một mảng biểu diễn heap, các phần tử từ chỉ số `heap_size / 2` trở đi là lá, nên ta cần duyệt từ `heap_size / 2 - 1` xuống đến 0.

32. Hãy điền vào `Code2`:

    a) heapify(0); b) heapify(1); c) heapify(i); d) heapify(i-1);

    Đáp án: A - **Giải thích:** Sau mỗi lần hoán đổi phần tử lớn nhất (ở đỉnh heap – `harr[0]`) với phần tử cuối cùng, ta cần gọi lại `heapify(0)` để phục hồi tính chất max-heap cho phần tử mới được đưa lên đỉnh.